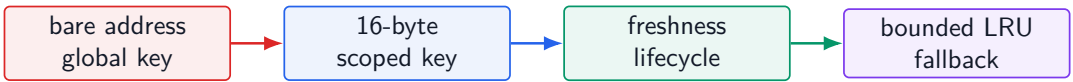
Phase 2

Lock Identity 与状态安全

让“同一把锁”的命名、生命周期和容量退化保持可解释



identity、调度隔离和 freshness 分离，避免把所有风险塞进一个 key

完成依据：static/BTF smoke、same-address alias、address reuse、1,152-lock map pressure 与当前 MariaDB 回归。

一句话结论：Phase 2 解决污染风险，不承诺 generation tracking

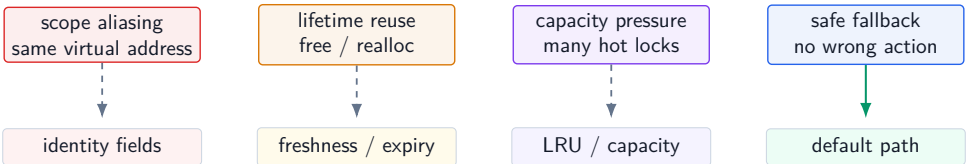
16 B	1024	1152	0
lock key size	LockNest LRU capacity	contended locks tested	map update failure

完成：private futex 使用 {tgid,uaddr} scope；kernel mutex 使用独立 kind；stale / LRU / cpumask 失败均退回默认路径。

边界：timestamp 只控制 freshness，不检测 free/realloc generation；地址复用的影响被限制为有界 advisory loss。

1,152-lock pressure controlled run 正常结束；LRU 淘汰造成的是 warm-up / locality loss，不是跨 key 状态复用。

审稿人担忧可以拆成三个正交问题



Phase 2 的目标是：任何 identity / lifetime / capacity 异常最多损失收益，不能制造跨锁 placement 或错误 boost。

三层边界：identity、isolation、freshness 不混为一谈

identity

$$K = \langle kind, addr, scope \rangle$$

回答：是不是同一把锁？

isolation

$$cpu \in p \rightarrow cpus_ptr$$

回答：这个 anchor CPU 是否允许？

freshness

$$now - last_update \leq expiry$$

回答：旧状态还能不能当 hint？



Key schema：所有 lock state 共享同一个最小身份结构



kernel mutex：`{KERNEL_MUTEX,kaddr,0}`；private futex：`{PRIVATE_FUTEX,uaddr,tgid}`。

固定 16 bytes：identity 足够表达，避免把 cgroup、时间和策略字段塞入 key。

cgroup 是调度隔离约束，不是锁身份；同一进程内的锁不会因 cgroup migration 被人为拆开。

Key 构造公式：同地址不代表同一锁

private futex

$$K_P(uaddr, tgid) = \langle P, uaddr, tgid \rangle$$

$$tgid_1 \neq tgid_2 \Rightarrow K_P(uaddr, tgid_1) \neq K_P(uaddr, tgid_2)$$

普通 private futex 的作用域是 address space / process scope；使用 tgid 阻断常见跨进程同地址混叠。

kernel mutex

$$K_K(kaddr) = \langle K, kaddr, 0 \rangle$$

内核地址已由内核地址空间命名，不与用户虚拟地址 key 混用。

kind 字段阻断“同一个数值地址、不同来源语义”的混叠。

代码修复 1：从裸 u64 address 到 typed scoped key

旧 prototype

```
u64 lock_addr;
lock_owners[lock_addr];
lock_nests[lock_addr];
waiters[lock_addr];
```

同一个用户虚拟地址在两个进程中可以表示不同同步对象。

当前实现

```
struct lock_key {
  u64 addr;
  u32 scope_id;
  u32 kind;
};
lock_nests[lock_key];
```

所有 owner、waiter、nest state 共享 key schema；跨进程 alias 被隔离。

$$\text{identity safety} = \text{typed source} + \text{scope} \quad \text{(not bare address)}$$

Same-address alias：父子进程观察到两个 scope



controlled runtime 的 alias case 产生两个不同 scope；两个进程不会共享 LockNest / LockBoost state。

这解决的是跨进程地址混叠，不是 free/realloc 后的 generation tracking。两类风险分开处理。

为什么 cgroup 不进入 lock identity ?

错误的耦合

$$K = \langle addr, tgid, cgroup \rangle$$

- 同一进程迁移 cgroup 后被切成两把锁。
- lock handoff stream 被隔离策略打断。
- identity 变成调度配置的函数。

当前分层

$$K = \langle kind, addr, scope \rangle$$

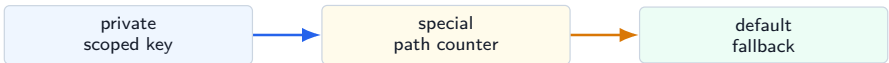
$$placement\ allowed \iff cpu \in p \rightarrow cpus_ptr$$

- cgroup 参与 accounting / budget，而非 identity。
- anchor CPU 不允许时直接 fallback。

身份回答“是不是同一把锁”；cpumask / cgroup 回答“这个动作是否允许”。

Shared、PI、requeue：明确边界比伪支持更安全

family	为什么不能复用 private key	当前实现	claim
shared futex	真实 key 涉及 mapping / inode / offset	classify + fallback	unsupported
PI futex	kernel PI owner / priority semantics different	classify + fallback	unsupported
requeue	waiter 可能从一地址迁移到另一地址	classify + fallback	unsupported
ordinary private	{tgid,uaddr} 足够表达常见作用域	key + hint / resolver	supported profile



Phase 2 不在 fast path 中重建完整 futex subsystem；只保证特殊路径不会污染 ordinary private-futex state。

地址复用：timestamp 能做什么，不能做什么

能够做

$$fresh(L, t) \iff t - last_update(L) \leq expiry$$

- 旧 anchor 超过 expiry 后不再参与 placement。
- stale entry 被删除，下一次事件重新 warm up。
- LRU 限制长期无人访问的状态规模。

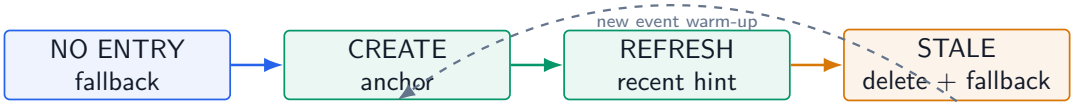
不能做

$$timestamp \neq generation$$

- 不观察 allocator free/realloc。
- 不知道同一地址是否已变成新 mutex。
- 不提供严格的 stale-free correctness guarantee。

因此论文 claim 是 “bounded advisory impact”，不是 “彻底解决地址复用”。

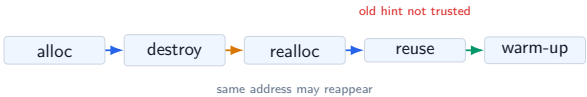
状态生命周期：stale 只允许丢收益，不允许继承强动作



owner state 在 acquire/unlock 更新；waiter state 在 exit/acquire/select_cpu 清理；anchor state 由 expiry + LRU 控制。

$$\textit{stale or invalid anchor} \Longrightarrow \textit{delete}(\textit{state}) \; + \; \textit{default_select_cpu}$$

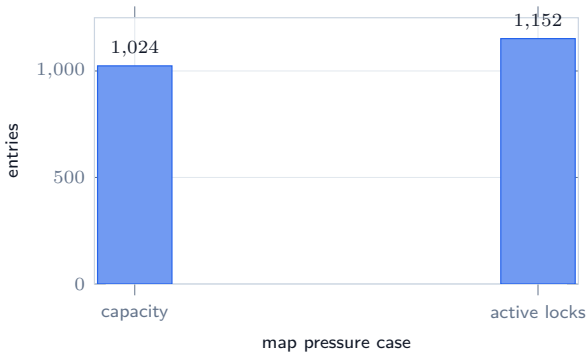
地址复用 controlled experiment：五轮 destroy / reinit



observation	result
destroy/reinit rounds	5
stale nest	4
owner boost from stale reuse	0
map update failure	0
benchmark return	0
scheduler state	disabled

实验支持“旧 advisory 状态被限制并重新 warm up”；不支持“系统检测到 allocator generation”。

Map pressure：1,024 项 bounded LRU 的安全退化



runtime policy

- map type: `BPF_MAP_TYPE_LRU_HASH`
- capacity: 1024
- test: 1,152 contended mutexes
- eviction: remove old entry, never reuse key
- miss: default scheduling

超过容量允许 anchor miss / warm-up，不能跨 key 继承 anchor。

Map pressure 结果：graceful degradation，而不是错误状态

metric	measured	expected boundary	interpretation
active contended mutexes	1,152	$>$ 1,024	pressure exercised
LRU capacity	1,024	fixed	bounded state
map update failure	0	should be 0	no ordinary HASH-full failure
cross-key state reuse	0 observed	must be 0	identity safety
benchmark return code	0	0	application completed
scheduler return code	0	0	scheduler completed
sched_ext after run	disabled	disabled	cleanup complete



代码修复 2：容量、admission 和 fallback 一起定义

旧路径

```
HASH[lock_addr] = anchor;
select_cpu(anchor);
// no scope
// no expiry
// no cpumask gate
```

满表、跨进程同址、过期 anchor 都可能直接污染 placement。

当前路径

```
key = {kind, addr, scope};
if (!fresh(key)) delete + fallback;
if (!cpumask_test(anchor)) fallback;
LRU[key] = anchor;
else default_select_cpu();
```

状态安全优先：缺失 anchor 只丢 locality，不制造错误调度动作。

安全不变量：异常只能让收益下降

key separation

$$K_a \neq K_b \Longrightarrow state[K_a] \text{ never read as } state[K_b]$$

freshness gate

$$now - last_update(K) > expiry \Longrightarrow delete(K) \wedge fallback$$

placement gate

$$anchor \notin cpus_ptr(p) \Longrightarrow default_select_cpu(p)$$

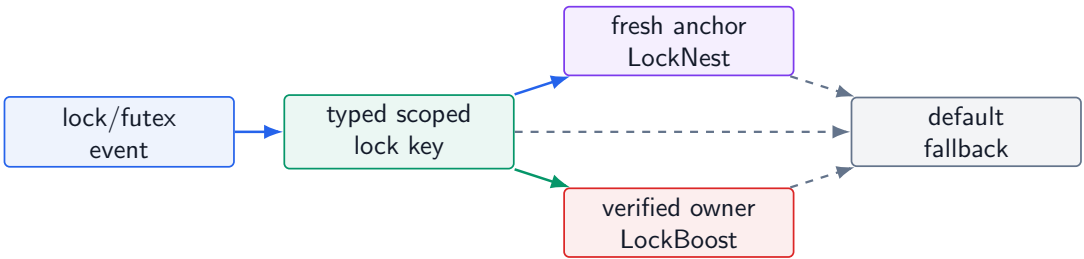
Phase 2 的 correctness claim 是 monotonic degradation：identity 不安全时不允许把错误状态升级成强动作。

三类 state 的 owner、生命周期和退化方式

state	key	producer	invalidation	failure meaning
`lock_owners`	lock_key	kernel acquire / verified owner	unlock / stale	no owner boost
`futex_hints`	lock_key	ordinary wake	expiry / refresh	no advisory anchor
`waiters`	TID	wait entry / resolver	exit / acquire / select_cpu	no placement relation
`lock_nests`	lock_key	admission / refresh	expiry / LRU	locality fallback
`contended_tasks`	owner TID	verified waiter	decay / unlock	no boost



Phase 1/2 组合路径：identity 修复不改变证据分层



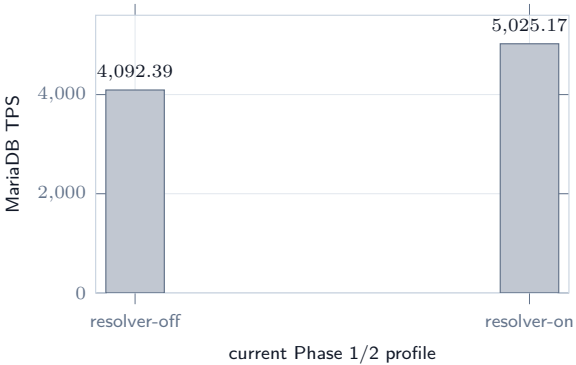
Phase 2 只改变“状态属于哪把锁、何时失效、容量不足如何退化”，不把 cgroup 或 timestamp 混入 owner evidence。

Controlled correctness matrix：每个风险都有对应实验

风险	controlled case	关键观察	结论
same-address alias	parent/child same uaddr	two distinct scopes	no cross-process key sharing
address reuse	destroy/reinit 5 rounds	4 stale nest, 0 boost	bounded advisory impact
shared futex	shared mapping path	fallback counter	no private-key pollution
PI futex	PI lock path	fallback counter	no owner claim
requeue	wake/requeue path	fallback counter	no state migration claim
map pressure	1,152 locks	map failure=0	LRU graceful degradation
cpumask	task restricted CPUs	anchor check + fallback	no illegal placement

这些实验验证的是 identity 和状态安全不变量，不是 16/64/256/1024/4096 热锁下的收益曲线。

当前版本性能 gate：Phase 1/2 修复没有裂化主线



profile	TPS	p99
resolver-off	4092.39	–
resolver-on	**5025.17**	**137.35 ms**

gate	result
scheduler	return 0
benchmark	return 0
stall / error	none
final state	disabled

该结果是 regression gate；正式性能结论仍使用后续 Phase 3/4 的统计实验。

优化对比：Phase 2 把复杂风险放在回退边界之外

维度	旧 prototype	当前实现	影响
identity	bare u64 address	kind + addr + scope	no cross-process alias
cgroup	mixed into state/key	isolation at placement/accounting	no identity fragmentation
lifetime	implicit timestamp	explicit expiry + delete + warm-up	bounded stale impact
capacity	ordinary HASH assumption	1,024-entry LRU	graceful pressure loss
CPU target	direct anchor	cpumask checked anchor	no illegal placement
unsupported paths	implicit behavior	classified fallback counters	claim boundary auditable

关键优化不是增加更多判断，而是把每个风险映射为一个 cheap gate：scope、freshness、capacity、cpumask。

Phase 2 的论文 claim 边界

可以写

- private futex 使用 scoped key 避免普通跨进程同址混叠；
- stale / LRU / cpumask 异常退化为 default path；
- 1,152-lock pressure 没有 map update failure 或错误 boost；
- identity、isolation、freshness 被明确分层。

不能写

- timestamp 已经检测 free/realloc generation；
- shared futex 使用完整 inode/offset key；
- 1024 项 map 下收益完全不下降；
- identity safety 等于长期公平性或性能保证。

推荐论文表述：scoped identity with bounded stale-state impact and graceful map-pressure fallback。

Phase 2 完成依据与后续工作

验收项	状态	依据
typed scoped key	PASS	kernel/private futex kind 与 scope 已统一
same-address alias	PASS	parent/child 产生不同 scope
unsupported futex boundary	PASS	shared/PI/requeue classify + fallback
address reuse	PASS (bounded claim)	5 轮 reuse，stale 清理，0 stale boost
LRU map pressure	PASS	1,152 locks，map update failure=0
placement isolation	PASS	cpumask gate + default fallback
performance gate	PASS	current MariaDB single-case 正常结束



Phase 2 结束后，LockNest/LockBoost 才有资格进入自适应收益优化；跨硬件、开销和直接 handoff 证据留给后续阶段。

结束：身份安全让收益可以被信任

Phase 2：scope before state

typed key、freshness gate、LRU fallback、monotonic degradation

